

Standard
Optimal Alg

SOA e Predição com Ajuda de Experts

MAC5005 - Fundamentos Matemáticos do Aprendizado Sequencial e Online

Victor S. Portella

IME-USP

2026-03-02

Relembrando - Classificação Binária Online

Para cada rodada $t = 1, \dots, T$:

- O **Adversário** escolhe um $x_t \in \mathcal{X}$
- O **Jogador/Algoritmo** escolhe uma predição $p_t \in \mathcal{Y} = \{0, 1\}$
- O **Adversário** escolhe e revela o rótulo real $y_t \in \{0, 1\}$

Relembrando - Classificação Binária Online

Para cada rodada $t = 1, \dots, T$:

- O **Adversário** escolhe um $x_t \in \mathcal{X}$
- O **Jogador/Algoritmo** escolhe uma predição $p_t \in \mathcal{Y} = \{0, 1\}$
- O **Adversário** escolhe e revela o rótulo real $y_t \in \{0, 1\}$

Assim como no aprendizado estatístico, vamos começar pelo **caso realizável**:

Temos uma classe de hipóteses $\mathcal{H} \subseteq \mathcal{Y}^{\mathcal{X}}$ conhecida, e existe $h^* \in \mathcal{H}$ tal que

$$y_t = h^*(x_t) \quad \text{para todo } t \in [T] := \{1, \dots, T\}.$$



Relembrando - Classificação Binária Online

Para cada rodada $t = 1, \dots, T$:

- O **Adversário** escolhe um $x_t \in \mathcal{X}$
- O **Jogador/Algoritmo** escolhe uma predição $p_t \in \mathcal{Y} = \{0, 1\}$
- O **Adversário** escolhe e revela o rótulo real $y_t \in \{0, 1\}$

Assim como no aprendizado estatístico, vamos começar pelo **caso realizável**:

Temos uma classe de hipóteses $\mathcal{H} \subseteq \mathcal{Y}^{\mathcal{X}}$ conhecida, e existe $h^* \in \mathcal{H}$ tal que

$$y_t = h^*(x_t) \quad \text{para todo } t \in [T] := \{1, \dots, T\}.$$

Importante: não assumimos mais nada sobre o adversário

Relembrando - Classificação Binária Online

Para cada rodada $t = 1, \dots, T$:

- O **Adversário** escolhe um $x_t \in \mathcal{X}$
- O **Jogador/Algoritmo** escolhe uma predição $p_t \in \mathcal{Y} = \{0, 1\}$
- O **Adversário** escolhe e revela o rótulo real $y_t \in \{0, 1\}$

Assim como no aprendizado estatístico, vamos começar pelo **caso realizável**:

Temos uma classe de hipóteses $\mathcal{H} \subseteq \mathcal{Y}^{\mathcal{X}}$ conhecida, e existe $h^* \in \mathcal{H}$ tal que

$$y_t = h^*(x_t) \quad \text{para todo } t \in [T] := \{1, \dots, T\}.$$

Importante: não assumimos mais nada sobre o adversário

Pergunta Qual o número máximo de erros que o algoritmo pode fazer?

Relembrando - Algoritmos para $\mathcal{H}$ finito

$$\mathcal{V}_t := \{h \in \mathcal{H} : h(x_s) = y_s \text{ para todo } s < t\}$$

Na rodada $t \in [T]$, O algoritmo **CONSISTENTE**

- Escolhe $h_t \in \mathcal{V}_t$
- Define $p_t = h_t(x_t)$

Na rodada $t \in [T]$, O algoritmo **HALVING**

- Escolhe $p_t \in \{0, 1\}$ que maximiza o tamanho do conjunto

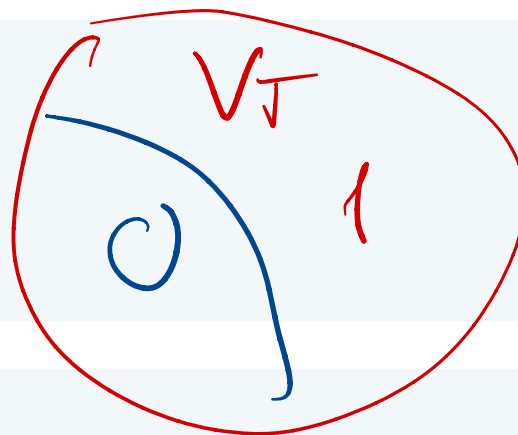
$$\{h \in \mathcal{V}_t : h(x_t) = p_t\}$$

Relembrando - Algoritmos para $\mathcal{H}$ finito

$$\mathcal{V}_t := \{h \in \mathcal{H} : h(x_s) = y_s \text{ para todo } s < t\}$$

Na rodada $t \in [T]$, O algoritmo **CONSISTENTE**

- Escolhe $h_t \in \mathcal{V}_t$
- Define $p_t = h_t(x_t)$



Na rodada $t \in [T]$, O algoritmo **HALVING**

- Escolhe $p_t \in \{0, 1\}$ que maximiza o tamanho do conjunto

$$\{h \in \mathcal{V}_t : h(x_t) = p_t\}$$

Teorema. Para p_t dado por **CONSISTENTE** e **HALVING** (resp.), temos

$$\sum_{t=1}^T \mathbf{1}[p_t \neq y_t] \leq |\mathcal{H}| - 1 \quad \text{e} \quad \sum_{t=1}^T \mathbf{1}[p_t \neq y_t] \leq \log_2(|\mathcal{H}|)$$

Relembrando - Árvores de estratégia

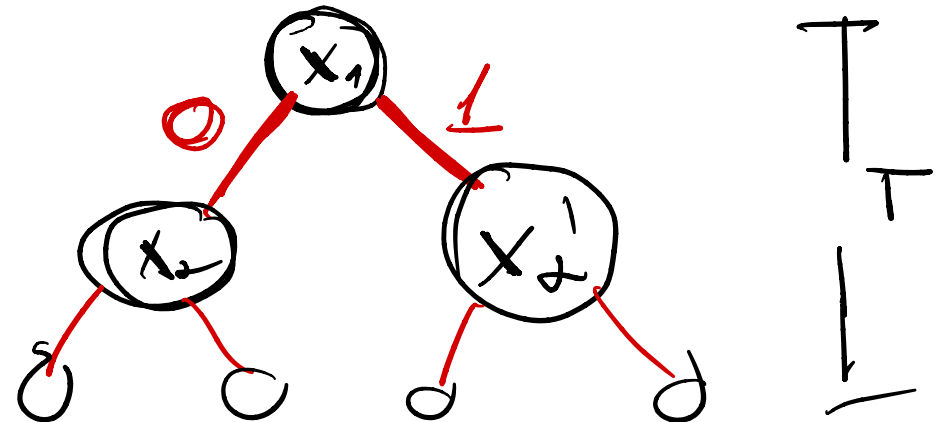
Uma **árvore de estratégia** (do adversário) é uma árvore binária completa de altura T com conjunto de vértices $v_1, \dots, v_{2^{T+1}-1} \in \mathcal{X}$ com v_1 de raiz.

Arestas e Notação:

- $\text{DESC}(v_i, 0) := v_{2i}$ (filho esquerdo de v)
- $\text{DESC}(v, 1) := v_{2i+1}$ (filho direito de v)

As escolhas do **adversário** são dadas pela árvore. Começando pela raiz $v = v_1$

- **Adversário** escolhe $x_t = v$;
- **Adversário** espera p_t , e define $y_t = |1 - p_t|$;
- **Adversário** atualia $v = \text{DESC}(v, y_t)$.



Relembrando - Dimensão de Littlestone

Uma árvore de estratégia com vértices $(v_1, \dots, v_{2^{T+1}-1})$ é fragmentada por $\mathcal{H}$ se **para todo** $y \in \{0, 1\}^T$, definindo $x_1 = v_1$ e

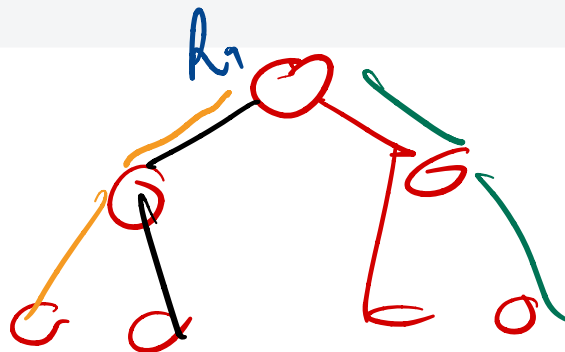
$$x_{t+1} = \text{DESC}(x_t, y_t) \quad \text{para todo } t \in [T-1],$$

existe $h \in \mathcal{H}$ tal que $h(x_t) = y_t$ para todo $t \in [T]$.

Assim como em aprendizado estatístico, a ideia de **fragmentação** nos leva a uma noção de **dimensão de $\mathcal{H}$**

A **dimensão de Littlestone** $\text{LDim}(\mathcal{H})$ de $\mathcal{H}$ é o maior $T \in \mathbb{N}$ tal que existe uma árvore de estratégia de profundidade T fragmentada por $\mathcal{H}$

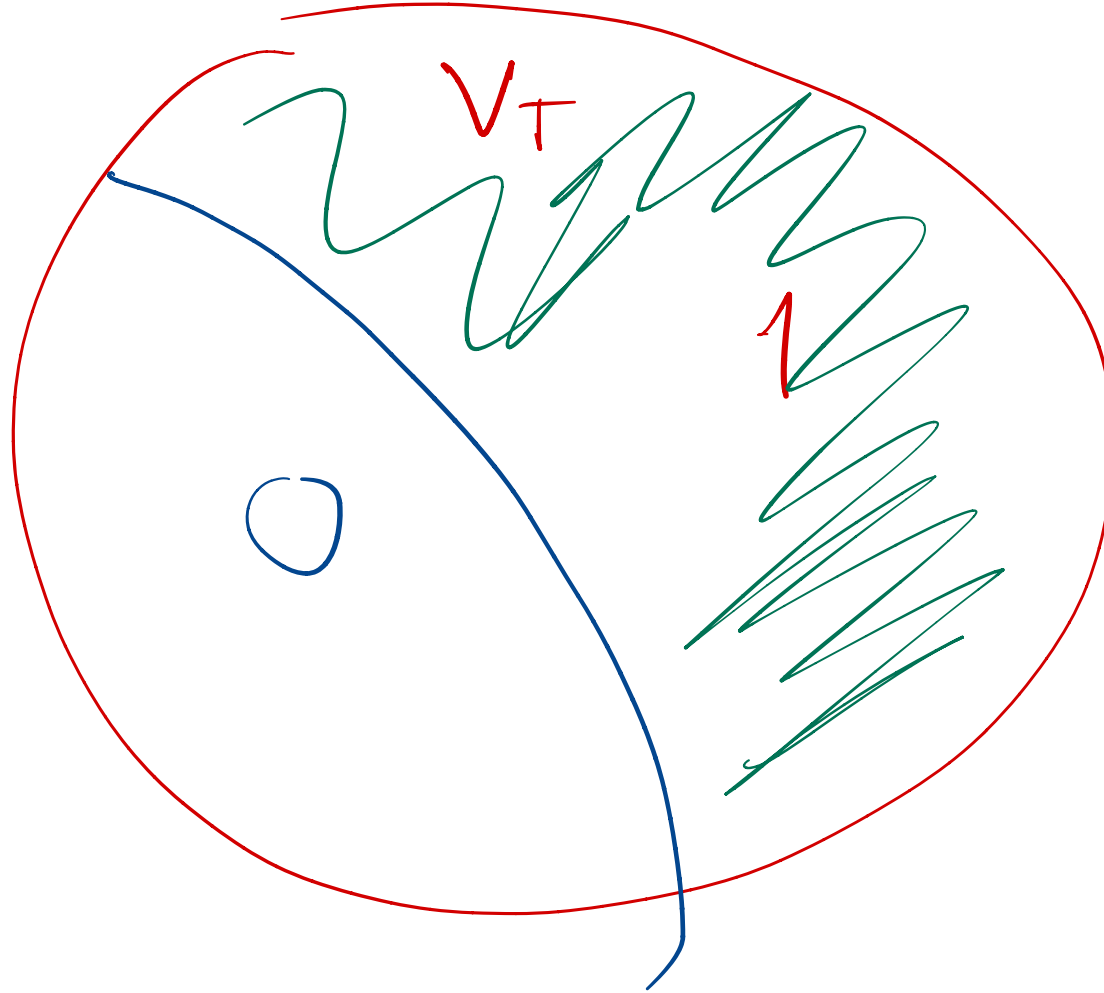
Lema. Nenhum algoritmo pro jogador contra um adversário realizado por $\mathcal{H}$ pode garantir menos que $\text{LDim}(\mathcal{H})$ erros.



Um Algoritmo **Ótimo** para Classificação Binária Online

Standard Optimal Algorithm (SOA)

O algoritmo ótimo para classificação binária online é idêntico ao **HALVING**



Standard Optimal Algorithm (SOA)

O **algoritmo ótimo** para classificação binária online é idêntico ao **HALVING**

Na rodada $t \in [T]$, o algoritmo **SOA**

- Escolhe $p_t \in \{0, 1\}$ que ~~maximiza o tamanho do conjunto~~ maximiza a dimensão de Littlestone de

$$\{h \in \mathcal{V}_t : h(x_t) = p_t\}$$

Standard Optimal Algorithm (SOA)

O **algoritmo ótimo** para classificação binária online é idêntico ao **HALVING**

Na rodada $t \in [T]$, o algoritmo **SOA**

- Escolhe $p_t \in \{0, 1\}$ que ~~maximiza o tamanho do conjunto~~ maximiza a dimensão de Littlestone de

$$\{h \in \mathcal{V}_t : h(x_t) = p_t\}$$

Teorema. Para p_t dado por **SOA**,

$$\sum_{t=1}^T \mathbf{1}[p_t \neq y_t] \leq \text{LDim}(\mathcal{H})$$

Standard Optimal Algorithm (SOA)

O **algoritmo ótimo** para classificação binária online é idêntico ao **HALVING**

Na rodada $t \in [T]$, o algoritmo **SOA**

- Escolhe $p_t \in \{0, 1\}$ que ~~maximiza o tamanho do conjunto~~ maximiza a dimensão de Littlestone de

$$\{h \in \mathcal{V}_t : h(x_t) = p_t\}$$

Teorema. Para p_t dado por **SOA**,

$$\sum_{t=1}^T \mathbf{1}[p_t \neq y_t] \leq \text{LDim}(\mathcal{H})$$

Corolário. No caso realizável, existe um algoritmo que sofre no máximo um número finito de erros em classificação binária online **se e só se** $\text{LDim}(\mathcal{H}) < \infty$

Comparando $\text{LDim}(\mathcal{H})$ com $\text{VC}(\mathcal{H})$

Teorema. Para qualquer $\mathcal{H} \subseteq \mathcal{X}^{\{0,1\}}$, temos que

$$\text{VC}(\mathcal{H}) \leq \text{LDim}(\mathcal{H}).$$

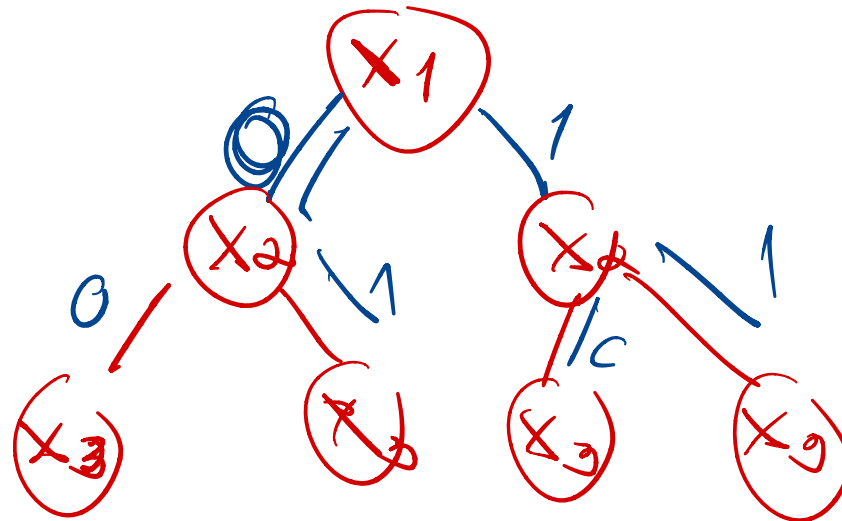
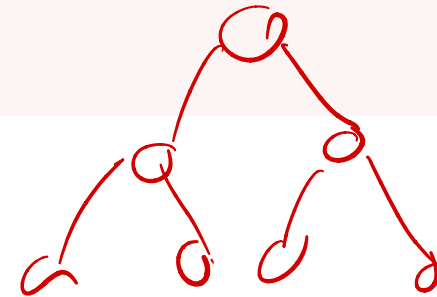
2

$+\infty$

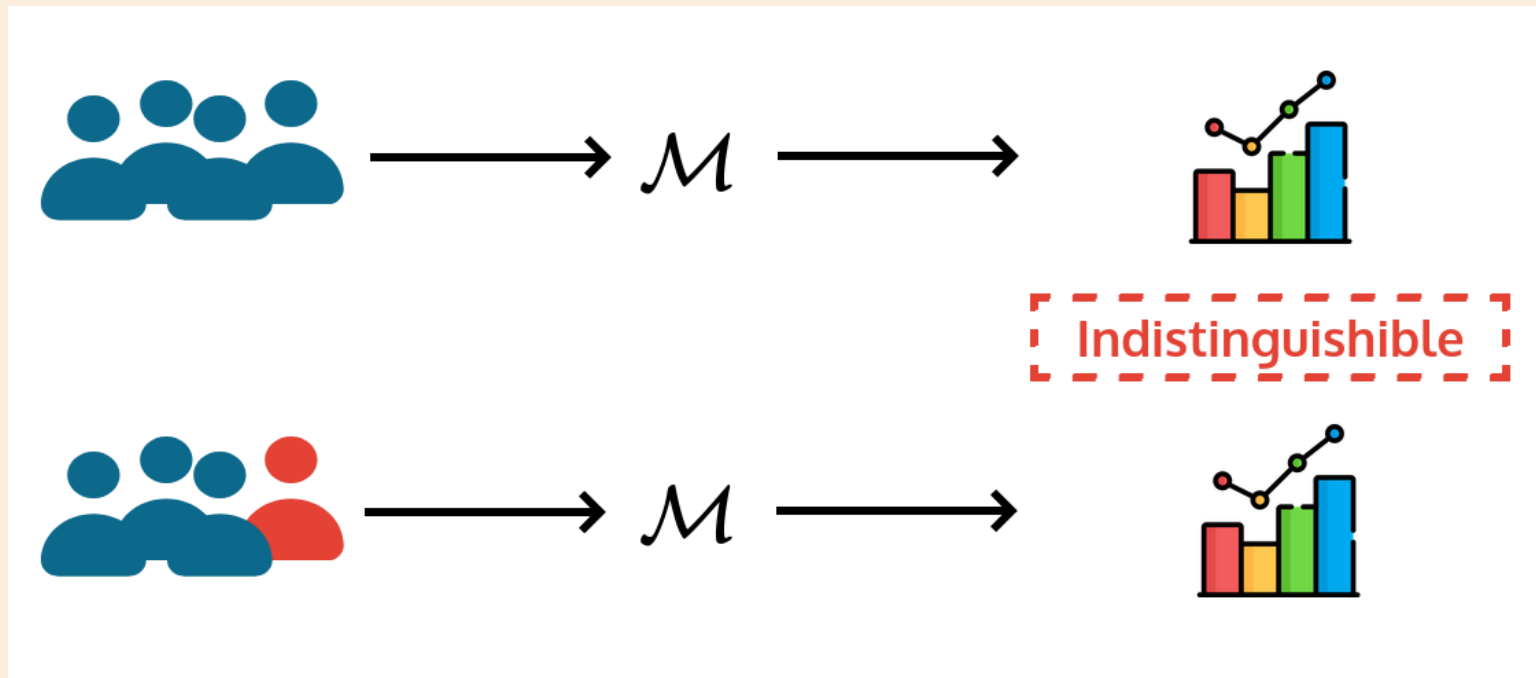
H fragmenta

0 1 0 1 0

$$S = (x_1, \dots, x_{VC})$$



Extra - Aprendizado Online e Privado



Private and Online Learnability Are Equivalent

NOGA ALON, Princeton University and Tel Aviv University

MARK BUN, Boston University

ROI LIVNI, Tel Aviv University

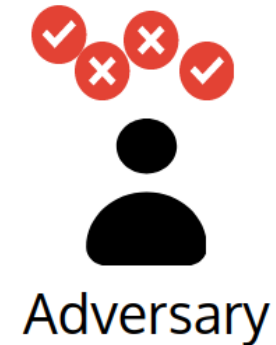
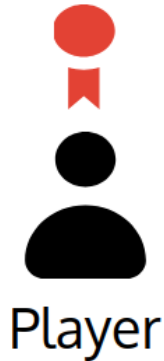
MARYANTHE MALLIARIS, University of Chicago

SHAY MORAN, Technion

Classificação Binária Online **Agnóstica** e Predição com Ajuda de Experts

Predição com Ajuda de Experts

Vamos estudar primeiro **Predição com Ajuda de Experts**

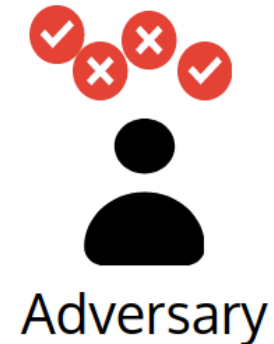


Para cada rodada $t = 1, \dots, T$:

- O **Jogador/Algoritmo** escolhe um expert $i_t \in [d]$ para seguir
- O **Adversário** escolhe custos $\ell_t \in [0, 1]^d$ para os experts
- O **Jogador/Algoritmo** sofre o custo $\ell_t(i_t)$) vê ℓ_t

Predição com Ajuda de Experts

Vamos estudar primeiro **Predição com Ajuda de Experts**



Para cada rodada $t = 1, \dots, T$:

- O **Jogador/Algoritmo** escolhe um expert $i_t \in [d]$ para seguir
- O **Adversário** escolhe custos $\ell_t \in [0, 1]^d$ para os experts
- O **Jogador/Algoritmo** sofre o custo $\ell_t(i)$

Nota. Classificação binária com $\mathcal{H}$ finito é um caso especial

Exemplo

Experts	Cumulative Loss		
	$t = 1$	$t = 2$	$t = 3$
	<u>1</u>	<u>1</u>	1
	<u>1</u>	0	0
	<u>1</u>	0	0.5
		2	

Medida de Desempenho: Regret

No pior caso, o jogador **erra toda rodada**.

Ou seja, numero de erros **não é informativo** nesse caso.

Medida de Desempenho: Regret

No pior caso, o jogador **erra toda rodada**.

Ou seja, numero de erros **não é informativo** nesse caso.

Alternativa - Regret: comparar custo com o do melhor expert do jogo.

$$\text{Regret}(T) = \sum_{t=1}^T \ell_t(i_t) - \min_{i \in [d]} \sum_{t=1}^T \ell_t(i)$$

Medida de Desempenho: Regret

No pior caso, o jogador **erra toda rodada**.

Ou seja, numero de erros **não é informativo** nesse caso.

Alternativa - Regret: comparar custo com o do melhor expert do jogo.

$$\text{Regret}(T) = \underbrace{\sum_{t=1}^T \ell_t(i_t)}_{0.6T} - \min_{i \in [d]} \underbrace{\sum_{t=1}^T \ell_t(i)}_{0.5T}$$

Objetivo: garantir regret sublinear no número de rodadas T : $0.5 \cdot T$

$$\frac{\text{Regret}(T)}{T} \longrightarrow 0 \quad \text{quando} \quad T \rightarrow \infty.$$

Medida de Desempenho: Regret

No pior caso, o jogador **erra toda rodada**.

Ou seja, numero de erros **não é informativo** nesse caso.

Alternativa - Regret: comparar custo com o do melhor expert do jogo.

$$\text{Regret}(T) = \sum_{t=1}^T \ell_t(i_t) - \min_{i \in \Delta} \sum_{t=1}^T \ell_t(i)$$

Objetivo: garantir regret sublinear no número de rodadas T :

$$\frac{\text{Regret}(T)}{T} \longrightarrow 0 \quad \text{quando} \quad T \rightarrow \infty.$$

Má notícia: ainda muito difícil!

Proposição. (Impossibilidade de Cover) Para qualquer algoritmo, existem custos tais que

$$\text{Regret}(T) \geq T/2.$$

Solução: Aleatorização/Convexificação

Solução: aleatorizar a escolha do jogador

Equivalente a “**convexificar**” a escolha do jogador

Solução: Aleatorização/Convexificação

Solução: aleatorizar a escolha do jogador

Equivalente a “**convexificar**” a escolha do jogador

- O **Jogador/Algoritmo** escolhe uma **distribuição** $p_t \in \Delta_d$ sobre os experts, onde

$$\Delta_d = \left\{ p \in [0, 1]^d : \sum_{i=1}^d p_i = 1 \right\}$$

- O **Adversário** escolhe custos $\ell_t \in [0, 1]^d$ para os experts
- O **Jogador/Algoritmo** sofre o **custo esperado** $\ell_t^\top p_t$

$$\left. \begin{array}{l} \ell_t(1) \cdot p_t(1) \\ + \ell_t(2) \cdot p_t(2) \dots \\ + \ell_t(d) \cdot p_t(d) \end{array} \right\} \mathbb{E}[\ell_t(i_t)] \quad i_t \sim p_t$$

$$\text{Regret} \leq T^{0.9}$$

$$\text{Regret} \leq T^{1/2}$$

$$\text{Regret} \leq \log T$$

Solução: Aleatorização/Convexificação

Solução: aleatorizar a escolha do jogador

Equivalente a “**convexificar**” a escolha do jogador

- O **Jogador/Algoritmo** escolhe uma **distribuição** $p_t \in \Delta_d$ sobre os experts, onde

$$\Delta_d = \left\{ p \in [0, 1]^d : \sum_{i=1}^d p_i = 1 \right\}$$

- O **Adversário** escolhe custos $\ell_t \in [0, 1]^d$ para os experts
- O **Jogador/Algoritmo** sofre o **custo esperado** $\ell_t^\top p_t$

Nota. Note que, na interpretação de aleatorização da escolha do jogador, o adversário não tem acesso aos “bits aleatórios” do jogador antes de escolher os custos;

Método dos Pesos Multiplicativos/Hedge

Método dos Pesos Multiplicativos/Hedge

MWU(T, η):

Defina $w_1 := \mathbf{1}$

Para $t = 1, \dots, T$:

- **Jogador** revela

$$p_t = \frac{w_t}{\|w_t\|_1} \in \Delta_d \quad \text{com } \|w_t\|_1 = \sum_{i=1}^d w_t(i)$$

- **Adversário** revela os custos $\ell_t \in [0, 1]^d$ e **Jogador** sofre custo $\ell_t^\top p_t$
- **Jogador** atualiza os pesos p_{t+1} para a próxima rodada:

$$w_{t+1}(i) := w_t(i) \cdot \exp(-\eta \ell_t(i)) \quad \text{para todo } i \in [d]$$

Handwritten notes: $\eta \geq 0$, $\ell_t(i)$, $e^{-\eta}$

Método dos Pesos Multiplicativos/Hedge

MWU(T, η):

Defina $w_1 := \mathbf{1}$

Para $t = 1, \dots, T$:

- **Jogador** revela

$$p_t = \frac{w_t}{\|w_t\|_1} \in \Delta_d$$

- **Adversário** revela os custos $\ell_t \in [0, 1]^d$ e **Jogador** sofre custo $\ell_t^\top p_t$
- **Jogador** atualiza os pesos p_{t+1} para a próxima rodada:

$$w_{t+1}(i) := w_t(i) \cdot \exp(-\eta \ell_t(i)) \quad \text{para todo } i \in [d]$$

Teorema. Para $\eta > 0$, o algoritmo MWU(T, η) com $\eta = \sqrt{2 \log d / T}$ garante

$$\text{Regret}(T) \leq \sqrt{2T \ln d}.$$

Aplicação: Classificação com $\mathcal{H}$ finito

Podemos usar MWU para **classificação binária online** com $\mathcal{H}$ finito

Caso “**aleatorizado/convexificado**”, onde a cada rodada t :

- O **Jogador** escolhe faz uma predição $\hat{y}_t \in [0, 1]$
- O **Adversário** revela $y_t \in \{0, 1\}$ e o jogador sofre custo $|\hat{y}_t - y_t|$

Aplicação: Classificação com $\mathcal{H}$ finito

Podemos usar MWU para **classificação binária online** com $\mathcal{H}$ finito

Caso “**aleatorizado/convexificado**”, onde a cada rodada t :

- O **Jogador** escolhe faz uma predição $\hat{y}_t \in [0, 1]$
- O **Adversário** revela $y_t \in \{0, 1\}$ e o jogador sofre custo $|\hat{y}_t - y_t|$

O **regret** para classificação binária online com respeito a classe hipóteses $\mathcal{H}$ é

$$\text{Regret}(\mathcal{H}, T) = \sum_{t=1}^T |\hat{y}_t - y_t| - \inf_{h \in \mathcal{H}} \sum_{t=1}^T |h(x_t) - y_t|$$

Aplicação: Classificação com $\mathcal{H}$ finito

Podemos usar MWU para **classificação binária online** com $\mathcal{H}$ finito

Caso “**aleatorizado/convexificado**”, onde a cada rodada t :

- O **Jogador** escolhe faz uma predição $\hat{y}_t \in [0, 1]$
- O **Adversário** revela $y_t \in \{0, 1\}$ e o jogador sofre custo $|\hat{y}_t - y_t|$

O **regret** para classificação binária online com respeito a classe hipóteses $\mathcal{H}$ é

$$\text{Regret}(\mathcal{H}, T) = \sum_{t=1}^T |\hat{y}_t - y_t| - \inf_{h \in \mathcal{H}} \sum_{t=1}^T |h(x_t) - y_t|$$

Corolário. Se $\mathcal{H}$ é finito, usando MWU o jogador pode garantir

$$\text{Regret}(\mathcal{H}, T) \leq \sqrt{2T \ln(|\mathcal{H}|)}.$$